

05-程序分析与符号执行复习讲义

目录

1	05 程序分析与符号执行复习讲义
---	------------------

2

1 05 程序分析与符号执行复习讲义

对应资料：

- Slides/Slides/Ch9-1 Dataflow Analysis.pdf
- Slides/Slides/Ch9-2-Symbolic Execution.pdf
- Slides/Slides/Ch12-1-Interprocedure Analysis.pdf
- Slides/Slides/Ch12-2 Pointer Analysis.pdf
- Slides/Slides/handout/5 Dataflow Analysis.pdf
- Slides/Slides/handout/6 Symbolic Execution.pdf
- Slides/Slides/handout/7 Interprocedure Analysis.pdf
- Slides/Slides/handout/8 Pointer Analysis.pdf
- Review/compile.pdf
- Exam/2025.pdf 符号执行与指针别名题

1.1 一、考试目标

本章真题重点集中在：

数据流分析：方向、meet、transfer

敏感性定义：path / flow / context / field

符号执行：符号状态、路径条件、约束求解

SAT: CNF、Tseitin、equisatisfiable

指针分析：Andersen 约束与闭包

零基础顺序：

先背定义，再会写公式，最后做 Andersen 闭包。

1.2 二、数据流分析框架

数据流分析在 CFG 上反复传播事实。

基本记号：

`in[B]` : 基本块 B 入口处的事实
`out[B]` : 基本块 B 出口处的事实
`meet` : 多个前驱或后继的合并方式
`transfer`: 基本块内部对事实的改变

前向分析:

```

in[B] = meet(out[P]) for P in pred(B)
out[B] = transfer_B(in[B])

```

后向分析:

```

out[B] = meet(in[S]) for S in succ(B)
in[B] = transfer_B(out[B])

```

Worklist 模板:

```

初始化 in/out
把相关基本块放入 worklist
while worklist 非空:
    取出 B
    根据方向重新计算 B 的 in/out
    若结果变化, 把受影响的后继或前驱加入 worklist

```

1.3 三、三种经典数据流分析

可达定值 Reaching Definitions:

问题: 哪些定义能到达当前位置。
 方向: 前向
`meet`: union
 公式: $out[B] = gen[B] \cup (in[B] - kill[B])$
 初值: emptyset

可用表达式 Available Expressions:

问题: 哪些表达式在所有路径上已经计算过, 且操作数未被改写。
 方向: 前向
`meet`: intersection
 公式: $out[B] = gen[B] \cup (in[B] - kill[B])$
 初值: 全集, 入口 out 为空集

活跃变量 Live Variables:

问题: 哪些变量当前值在之后还会被使用。
 方向: 后向
`meet`: union
 公式: $in[B] = use[B] \cup (out[B] - def[B])$
 初值: emptyset

1.4 四、敏感性定义

直接背：

flow-sensitive:

区分程序点和语句顺序。

path-sensitive:

区分不同控制流路径。

context-sensitive:

区分函数调用上下文。

field-sensitive:

区分对象的不同字段。

答题补一句：

敏感性越强，分析精度越高，状态数量和计算成本越高。

1.5 五、符号执行

符号执行把输入当成符号，而不是具体值。

过程：

1. 输入变量赋符号值，例如 `x = alpha_x`。
2. 赋值语句更新符号状态。
3. 分支语句拆分路径。
4. 每条路径维护 `path condition`。
5. 用约束求解器判断 `path condition` 是否可满足。

例：

```
if x > 0:
    y = x + 1
else:
    y = -x
```

路径 1:

```
PC = alpha_x > 0
y = alpha_x + 1
```

路径 2:

```
PC = alpha_x <= 0
y = -alpha_x
```

优点:

- 路径敏感。
- 能生成测试输入。
- 能发现深层路径错误。

成本:

- 路径爆炸。
- 循环和递归需要限制或归纳。
- 外部函数需要建模。

1.6 六、CNF 与 SAT

文字:

x 或 $\text{not } x$

子句:

$(x \text{ or } \text{not } y \text{ or } z)$

CNF:

子句的合取。

$(x \text{ or } y) \text{ and } (\text{not } x \text{ or } z)$

可满足:

存在一组布尔赋值, 使公式为真。

同时可满足 equisatisfiable:

两个公式要么都可满足, 要么都不可满足。
它不要求两个公式在同一变量集上逻辑等价。

1.7 七、Tseitin 变换

目的:

把任意布尔公式线性规模转成 CNF, 并保持同时可满足。

步骤:

1. 给每个子表达式引入新变量。
2. 为新变量与子表达式的语义关系写 CNF。
3. 加入根变量为真的子句。

常用转换:

$x \leftrightarrow (a \text{ and } b)$
 $(\text{not } x \text{ or } a) \text{ and } (\text{not } x \text{ or } b) \text{ and } (x \text{ or not } a \text{ or not } b)$

$x \leftrightarrow (a \text{ or } b)$
 $(x \text{ or not } a) \text{ and } (x \text{ or not } b) \text{ and } (\text{not } x \text{ or } a \text{ or } b)$

$x \leftrightarrow \text{not } a$
 $(\text{not } x \text{ or not } a) \text{ and } (x \text{ or } a)$

真题证明模板:

$F1 = (a_0 \text{ or } \dots \text{ or } a_n \text{ or } c) \text{ and } (b_0 \text{ or } \dots \text{ or } b_m \text{ or not } c)$
 $F2 = (a_0 \text{ or } \dots \text{ or } a_n \text{ or } b_0 \text{ or } \dots \text{ or } b_m)$

若 F1 可满足:

若 c 为 **false**, 则第一项要求某个 a_i 为 **true**。

若 c 为 **true**, 则第二项要求某个 b_j 为 **true**。

因此 F2 可满足。

若 F2 可满足:

若某个 a_i 为 **true**, 令 $c = \text{true}$, 则 F1 两个子句都为真。

若某个 b_j 为 **true**, 令 $c = \text{false}$, 则 F1 两个子句都为真。

因此 F1 可满足。

所以 F1 与 F2 同时可满足。

1.8 八、DPLL 与 CDCL

DPLL:

选择变量赋值

单子句传播

发现冲突则回溯

直到找到可满足赋值或证明不可满足

CDCL:

在 DPLL 基础上加入冲突分析和学习子句。
学习子句用于剪枝，避免重复进入同类冲突。

1.9 九、Andersen 指针分析

Andersen 是包含约束的指针分析。

points-to 集合:

$\text{pts}(p) = p$ 能指向的对象集合

四类约束必须会写:

```

y = &x:
  x in pts(y)

y = x:
  pts(x) subset pts(y)

*y = x:
  for v in pts(y), pts(x) subset pts(v)

y = *x:
  for v in pts(x), pts(v) subset pts(y)

```

1.10 十、Andersen 真题最小例子

题面结构:

```

p = &a;
q = &b;
*p = q;
r = &c;
s = p;
t = *p;
*s = r;

```

逐句约束:

```

a in pts(p)
b in pts(q)
for v in pts(p), pts(q) subset pts(v)
c in pts(r)
pts(p) subset pts(s)
for v in pts(p), pts(v) subset pts(t)
for v in pts(s), pts(r) subset pts(v)

```

闭包推导：

```

pts(p) = {a}
pts(q) = {b}
由 *p = q 得 pts(a) 包含 {b}
pts(r) = {c}
由 s = p 得 pts(s) 包含 {a}
由 t = *p 且 pts(p) = {a} 得 pts(a) subset pts(t), 所以 pts(t) 包含 {b}
由 *s = r 且 pts(s) = {a} 得 pts(a) 包含 {c}
pts(a) = {b,c}
pts(t) = {b,c}

```

1.11 十一、自测题

题 1: 活跃变量分析是前向还是后向?

答案:

后向。

题 2: 可用表达式的 meet 是什么?

答案:

intersection。

题 3: 符号执行为什么路径敏感?

答案:

它为每条控制流路径维护独立的 path condition。

题 4: $y = *x$ 的 Andersen 约束怎么写?

答案:

```

for v in pts(x), pts(v) subset pts(y)

```